Pemrograman Berbasis Objek

RTI253007 | D-IV Teknik Informatika

Pertemuan 11: **SOLID Principle dan Collections**

Lima prinsip desain kelas yang baik

Yang Akan Kamu Pelajari

- Lima prinsip SOLID agar kelas mudah dipahami, diperluas, dan diuji
- Cara mengenali kelas yang melanggar salah satu prinsip itu
- Cara menyimpan data yang jumlahnya berubah-ubah (`ArrayList` dan `Map`)
- Kelima prinsip dalam satu sistem pemrosesan pesanan

> Latihan pemrograman untuk materi hari ini tersedia di jobsheet Praktikum Pemrograman Berbasis Objek (RTI253008), Pertemuan 11.

Peta Sesi Hari Ini

- **Sesi 1 (50')**: Single Responsibility dan Open/Closed
- **Sesi 2 (50')**: Liskov Substitution dan Interface Segregation
- **Sesi 3 (50')**: Dependency Inversion, lalu `ArrayList` dan `Map`
- **Sesi 4 (50')**: Studi kasus, sistem pemrosesan pesanan

Bagian 1

SOLID: Single Responsibility dan Open/Closed

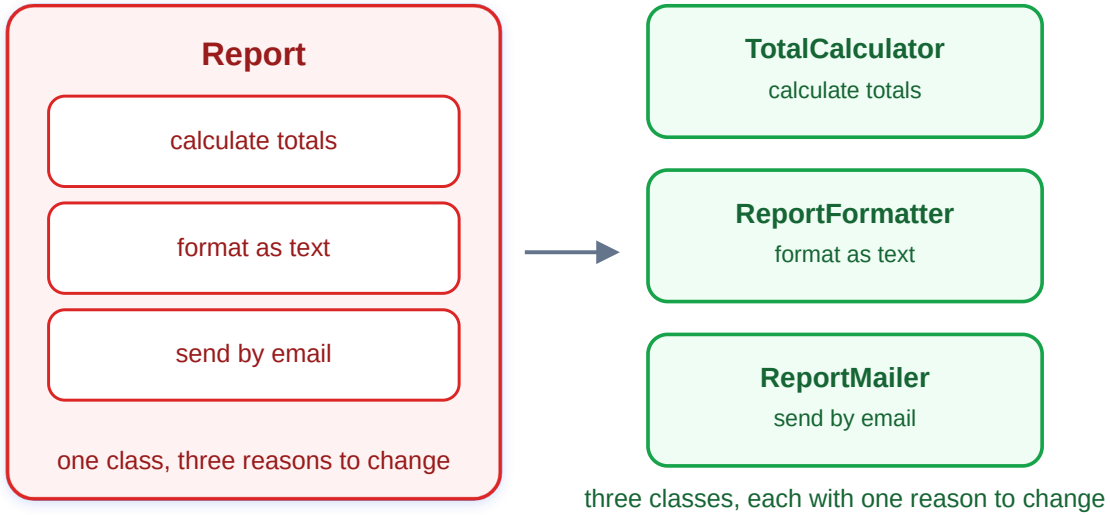
Sesi 1 dari 4

Lima Prinsip Desain Kelas yang Baik

> **SOLID** adalah lima prinsip agar kelas mudah dipahami, diperluas, dan diuji: **S**ingle Responsibility, **O**pen/Closed, **L**iskov Substitution, **I**nterface Segregation, **D**ependency Inversion.

Beberapa sudah kamu pakai tanpa sadar. Hari ini kita beri nama resminya.

Single Responsibility Principle (SRP)



Single Responsibility Principle: one class, one reason to change

Di restoran, koki, kasir, dan pelayan punya tugas masing-masing. Salah di dapur tidak mengacaukan pembayaran.

> Satu kelas punya satu tanggung jawab, yaitu satu alasan untuk berubah. **Satu kelas, satu tugas.**

Mengapa Ini Penting?

Kelas `Report` menghitung total, memformat tampilan, dan mengirim email sekaligus. Mengubah format tampilan bisa tanpa sengaja merusak perhitungan.

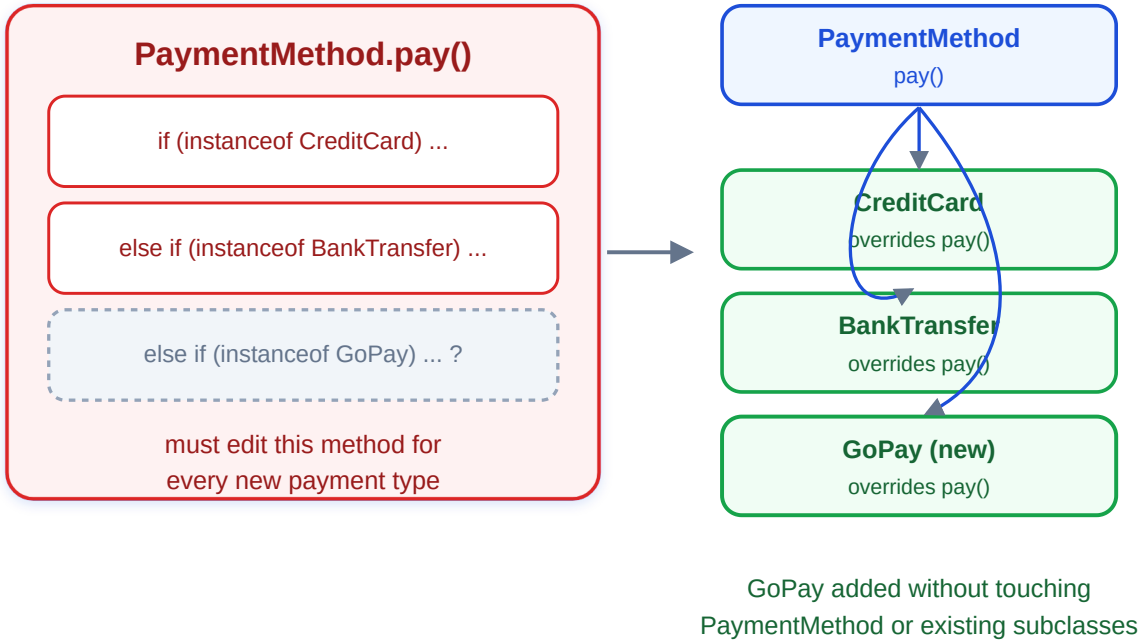
> Kelas dengan satu tugas aman diubah. Mengganti cara kirim email tidak menyentuh perhitungan.

Contoh Kode: `Report` Dipisah Jadi Tiga Kelas

```
class TotalCalculator { double calculate(Report r) { /* ... */ return 0; } }
class ReportFormatter { String format(Report r) { /* ... */ return ""; } }
class ReportMailer    { void send(Report r) { /* ... */ } }
```

1. Menghitung total: `TotalCalculator`.
2. Memformat tampilan: `ReportFormatter`.
3. Mengirim email: `ReportMailer`. Tiap kelas satu alasan untuk berubah.

Open/Closed Principle (OCP)



Stop kontak menerima alat apa pun. Alat baru tinggal dicolokkan, instalasi rumah tidak dibongkar.

> **Open/Closed Principle**: terbuka untuk diperluas, tertutup untuk diubah. **Kelas baru, bukan ubah kode lama.**

Mengapa Ini Penting?

Satu method besar berisi `if` / `else` untuk tiap jenis pembayaran. Menambah GoPay berarti menyisipkan cabang baru. Salah sedikit, cabang kartu kredit ikut rusak.

> Dengan kelas baru, kode lama yang sudah teruji tidak disentuh.

Contoh Kode: Menambah GoPay Tanpa Mengubah Kode Lama

```
// Sebelum: tiap jenis baru mengubah method ini
if (type.equals("CREDIT_CARD")) { chargeCard(amount); }
else if (type.equals("BANK_TRANSFER")) { transferBank(amount); }

// Sesudah: cukup kelas baru
class GoPay extends PaymentMethod {
    void pay(double amount) { /* logika GoPay */ }
}
```

`PaymentMethod`, `CreditCard`, dan `BankTransfer` tidak berubah satu baris pun.

Kesalahan Umum: Cabang `if` untuk Tiap Jenis Baru

> **Salah:** tiap jenis diskon baru menambah satu cabang di `calculateDiscount()`.

```
if (type.equals("REGULAR")) { return amount * 5 / 100; }
else if (type.equals("WHOLESALE")) { return amount * 20 / 100; }
else if (type.equals("STUDENT")) { return amount * 10 / 100; }   // cabang baru lagi
```

Benar: interface `DiscountPolicy` dengan `calculate(amount)`, satu kelas per jenis diskon. Jenis baru tidak menyentuh method lama.

Latihan

Prinsip mana yang dilanggar: Single Responsibility atau Open/Closed?

1. `UserManager` memvalidasi input, menyimpan ke database, dan mengirim email.
2. Jenis pembayaran baru mengharuskan menambah cabang `if` di `pay()`.
3. `InvoicePrinter` mencetak faktur sekaligus menghitung pajaknya.

Jawaban Latihan

1. **Single Responsibility.** Tiga tugas dalam satu kelas.
2. **Open/Closed.** Method lama diubah tiap ada jenis baru.
3. **Single Responsibility.** Mencetak dan menghitung pajak adalah dua tugas.

Rangkuman Bagian 1

- Single Responsibility: satu kelas, satu tugas.
- Open/Closed: kelas baru, bukan ubah kode lama.
- Perubahan kecil tetap kecil: hanya satu kelas disentuh.

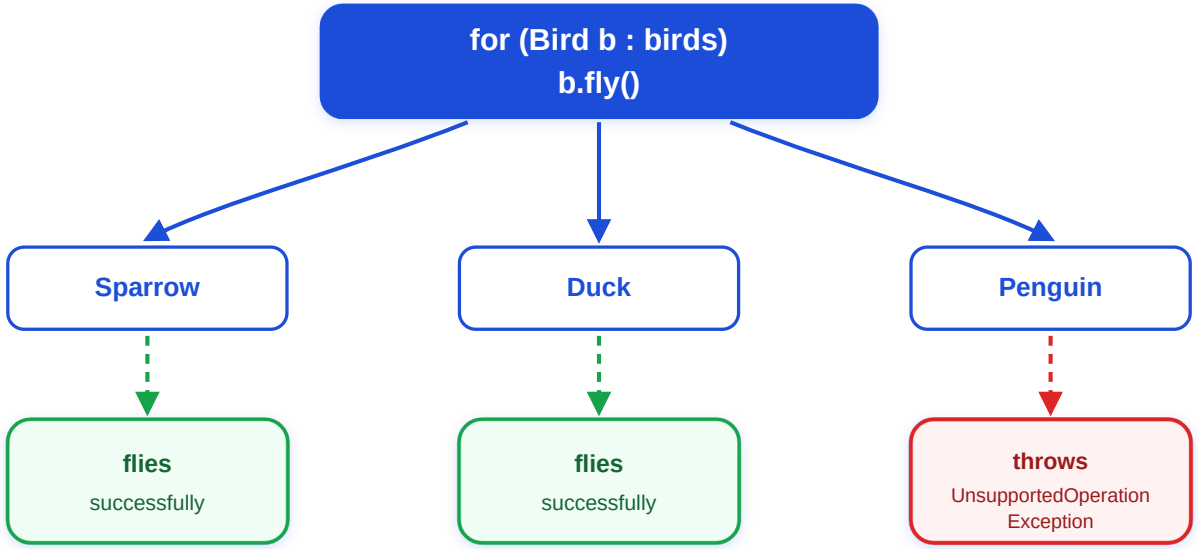
Selanjutnya: Liskov Substitution dan Interface Segregation.

Bagian 2

SOLID: Liskov Substitution dan Interface Segregation

Sesi 2 dari 4

Liskov Substitution Principle (LSP)



Penguin cannot substitute for Bird here,
it breaks a call site that works for every other subclass

Kamu memesan ojek, dan yang datang motor mana pun. Penggantinya harus tetap bisa mengantarmu.

> **Liskov Substitution Principle**: subclass harus bisa menggantikan superclass-nya tanpa membuat program salah. `Sparrow` dan `Duck` bisa, `Penguin` tidak.
> **Pengganti harus tetap bisa.**

Mengapa Ini Penting?

Kode yang memanggil lewat tipe superclass percaya pada janji superclass itu. Bila satu subclass mengingkari janji, program gagal jauh dari subclass itu.

> Compiler tidak menolaknya. Error baru muncul saat program berjalan, mungkin di tangan pengguna.

Kesalahan Umum: Subclass Mengingkari Janji

> **Salah:** `Penguin extends Bird` meng-override `fly()` untuk melempar `UnsupportedOperationException`.

```
for (Bird b : birds) {
    b.fly();   // error begitu b ternyata Penguin
}
```

Benar: `Penguin` tidak punya `fly()` sama sekali (slide berikut). Kode di atas mengira semua `Bird` bisa `fly()`.

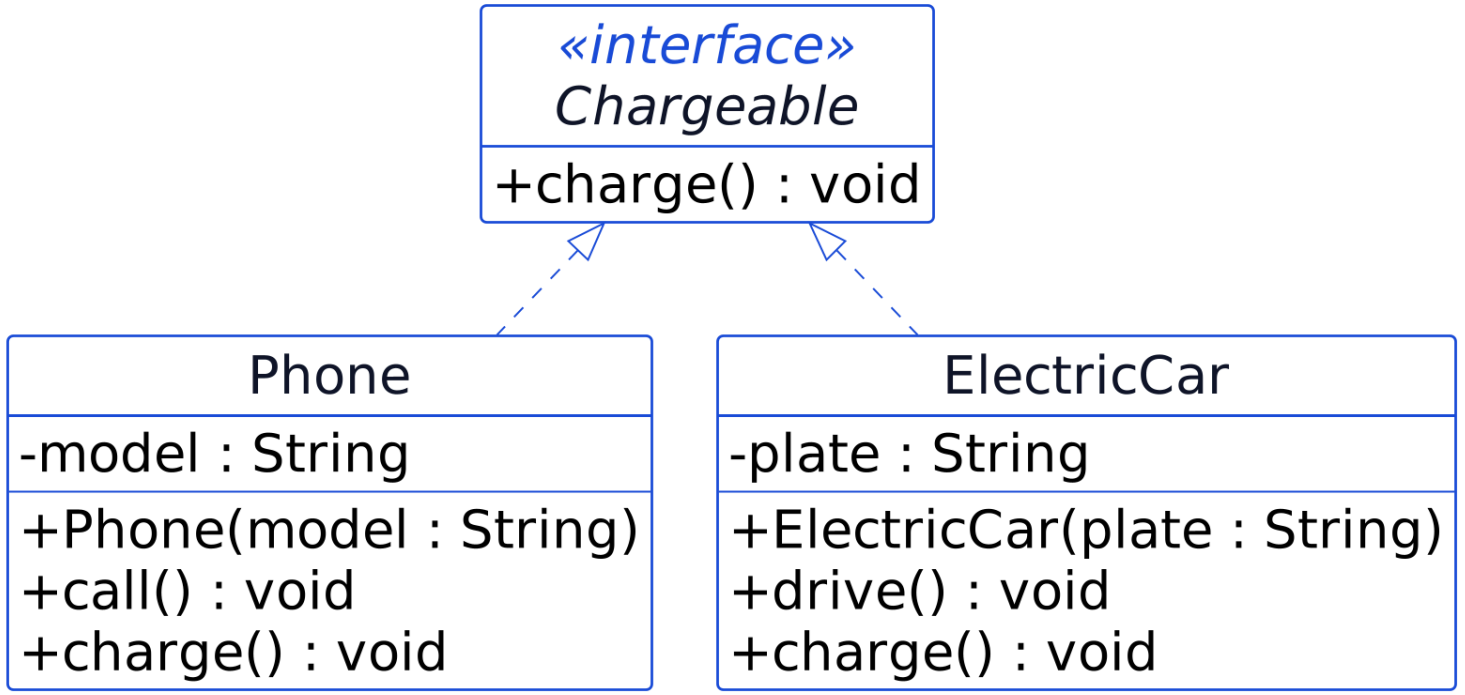
Perbaikan: Pisahkan Kemampuan Terbang

> Pindahkan `fly()` ke interface kecil `Flyable`. Hanya `Sparrow` dan `Duck` yang mengimplementasikannya.

```
interface Flyable { void fly(); }
class Sparrow extends Bird implements Flyable { ... }
class Penguin extends Bird { /* tidak implements Flyable */ }
```

Pemanggil memeriksa dulu: `if (b instanceof Flyable f) f.fly();`. Bentuk ini memberi variabel `f` bertipe `Flyable` tanpa cast. Pola ini muncul lagi di Bagian 4 (`Shippable`).

Interface Segregation Principle (ISP)



Remote TV yang hanya perlu menyalakan dan mematikan tidak butuh 50 tombol channel.

Interface Segregation Principle: interface kecil dan fokus, supaya kelas tidak dipaksa mengisi method yang tidak relevan. `Chargeable` di atas: satu method, dua kelas yang tidak berkerabat.

Interface kecil, satu kemampuan.

Mengapa Ini Penting?

Bila `charge()` dan `call()` digabung dalam satu interface `Device`, `ElectricCar` wajib menulis `call()`. Pilihannya dua dan keduanya buruk: kosong, atau melempar exception.

> Kode yang memegang `Device` mengira `call()` berfungsi. Interface yang gemuk ikut melanggar Liskov.

Contoh Kode: Memecah Interface yang Gemuk

```
// Sebelum: satu interface gemuk
interface Device { void charge(); void call(); }

// Sesudah: dua interface kecil
interface Chargeable { void charge(); }
interface Caller { void call(); }
class ElectricCar implements Chargeable { /* hanya charge() */ }
```

1. Sebelum: `ElectricCar` terpaksa menulis `call()`.
2. Sesudah: dua interface, satu kemampuan masing-masing.
3. `ElectricCar` hanya `implements Chargeable`.

Latihan

Prinsip mana yang dilanggar: Liskov Substitution atau Interface Segregation?

1. `Penguin extends Bird`, tetapi `fly()`-nya melempar exception.
2. `interface Printer` punya `print()`, `scan()`, `fax()`. `SimplePrinter` hanya bisa `print()`.
3. `ReadOnlyList extends ArrayList`, tetapi `add()`-nya melempar exception.

Jawaban Latihan

1. **Liskov Substitution.** `Penguin` mengingkari janji `Bird`.
2. **Interface Segregation.** `SimplePrinter` dipaksa mengisi `scan()` dan `fax()`.
3. **Liskov Substitution.** Pemakai `ArrayList` mengira `add()` selalu bisa.

Rangkuman Bagian 2

- Liskov Substitution: pengganti harus tetap bisa.
- Interface Segregation: interface kecil, satu kemampuan.
- Interface gemuk memaksa method palsu, dan itu melanggar Liskov.

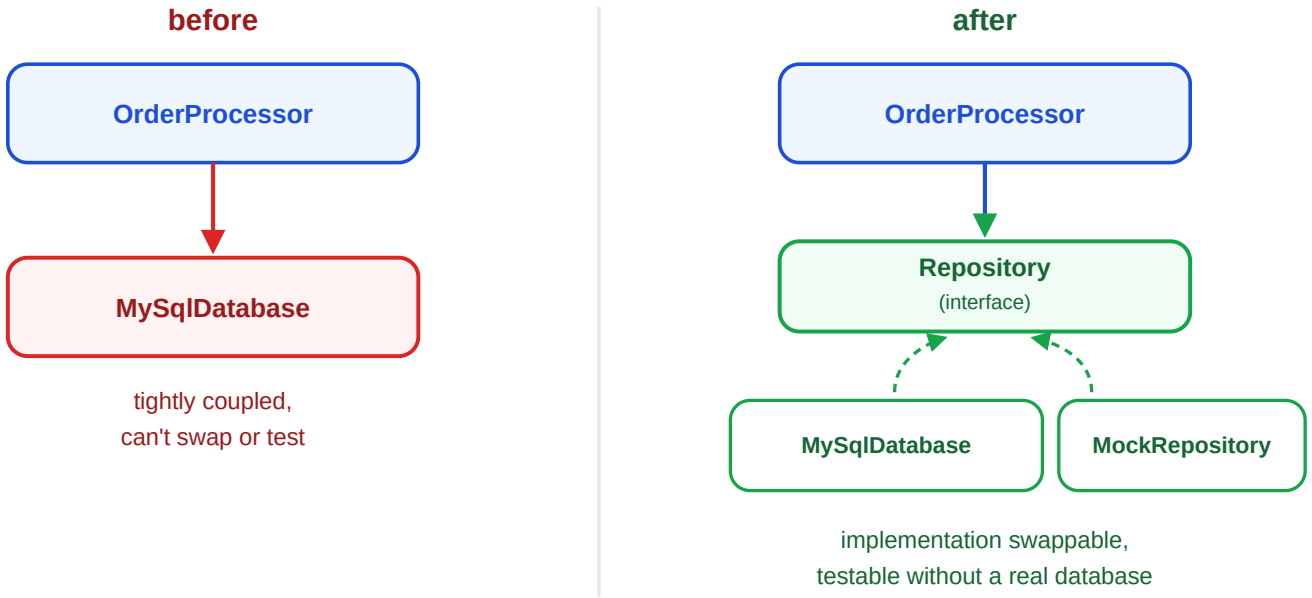
Selanjutnya: Dependency Inversion, lalu `ArrayList` dan `Map`.

Bagian 3

SOLID: Dependency Inversion, dan Collections

Sesi 3 dari 4

Dependency Inversion Principle (DIP)



Kasir hanya mengenal mesin kartu, bukan bank penerbitnya. Kartu bank mana pun bisa dipakai.

> **Dependency Inversion Principle**: kelas yang mengatur alur (`OrderProcessor`) bergantung pada interface, bukan pada kelas sungguhan (`MySqlDatabase`).
> **Bergantung pada interface, bukan kelas sungguhan.**

Mengapa Ini Penting?

`OrderProcessor` yang langsung memakai `MySqlDatabase` terikat ke MySQL. Pindah database atau menguji dengan tiruan menjadi sulit.

> Dengan interface `Repository`, penyimpanan bisa berganti tanpa mengubah `OrderProcessor`.

Contoh Kode: `OrderProcessor` Bergantung pada Interface

```
interface Repository { void save(Order order, int total); }

class OrderProcessor {
    private Repository repository;
    OrderProcessor(Repository repository) { this.repository = repository; }
}
```

1. `Repository` hanya menyatakan kemampuan `save(...)`.
2. `OrderProcessor` menerima `Repository` lewat constructor.
3. Kode yang membuat `OrderProcessor` memilih implementasinya.

`ArrayList` dan `Map`

`InMemoryRepository` harus menyimpan banyak `Order`. Array biasa terlalu kaku.

	Array	`ArrayList`	`Map`
Ukuran	tetap sejak dibuat	bertambah otomatis	bertambah otomatis
Cara ambil	lewat indeks	lewat indeks, `get(1)`	lewat kunci, `get("WHOLESALE")`
Cocok untuk	jumlah pasti	daftar berurutan	cari langsung lewat kunci

> Elemen di luar ukuran array: `ArrayIndexOutOfBoundsException`. **ArrayList** bertambah otomatis. **Map** mencari langsung lewat kunci; `LinkedHashMap` menjaga urutan penambahan.

Contoh Kode: `ArrayList` dan `Map`

```
List<Order> orders = new ArrayList<>();
orders.add(order);                                  // tidak ada batas ukuran

Map<String, DiscountPolicy> policies = new HashMap<>();
policies.put("WHOLESALE", new WholesaleDiscountPolicy());
DiscountPolicy policy = policies.get("WHOLESALE");   // langsung, tanpa looping
```

1. `List<Order>`: daftar yang hanya berisi `Order`, bertambah lewat `add(...)`.
2. `Map<String, DiscountPolicy>`: kunci jenis pelanggan, nilai policy (Bagian 1).
3. `get("WHOLESALE")` langsung. Kunci tidak ada: hasilnya `null`.

Latihan

1. `OrderProcessor` berisi `private MySqlDatabase db = new MySqlDatabase();`. Prinsip mana yang dilanggar?
2. Pilih `ArrayList` atau `Map`: daftar pesanan yang hanya dicetak berurutan.
3. Pilih `ArrayList` atau `Map`: data pelanggan yang sering dicari lewat nomor pelanggan.

Jawaban Latihan

1. **Dependency Inversion.** Terima `Repository` lewat constructor.
2. **`ArrayList`.** Hanya dibaca berurutan.
3. **`Map`.** Nomor pelanggan menjadi kunci.

Rangkuman Bagian 3

- Dependency Inversion: bergantung pada interface, bukan kelas sungguhan.
- `new` kelas sungguhan di dalam kelas tetap melanggarnya.
- `ArrayList` untuk daftar yang bertambah, `Map` untuk cari lewat kunci.

Selanjutnya: kelima prinsip dalam satu studi kasus.

Bagian 4

Studi Kasus: Kelima Prinsip dalam Satu Sistem

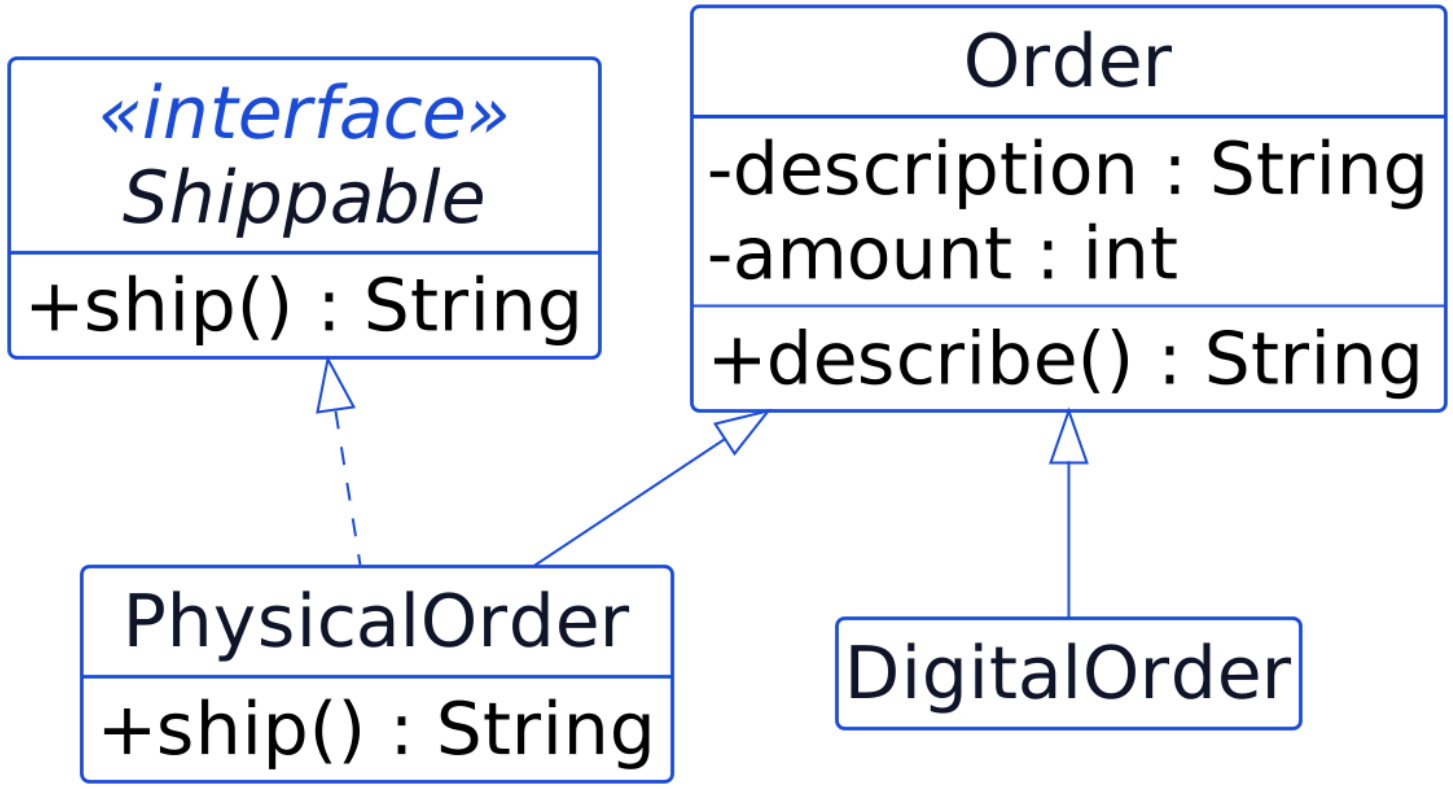
Sesi 4 dari 4

Satu Sistem Pemrosesan Pesanan

Toko daring punya pesanan fisik (dikirim) dan digital (tidak dikirim). Diskon berbeda tiap jenis pelanggan.

- `Order`, dengan subclass `PhysicalOrder` dan `DigitalOrder`
- `Customer`, punya jenis pelanggan (reguler, grosir)
- `OrderProcessor`, koordinator alur pesanan
- `DiscountCalculator` dan `DiscountPolicy`, menghitung diskon
- `Repository`, menyimpan pesanan; `ReceiptPrinter`, mencetak struk

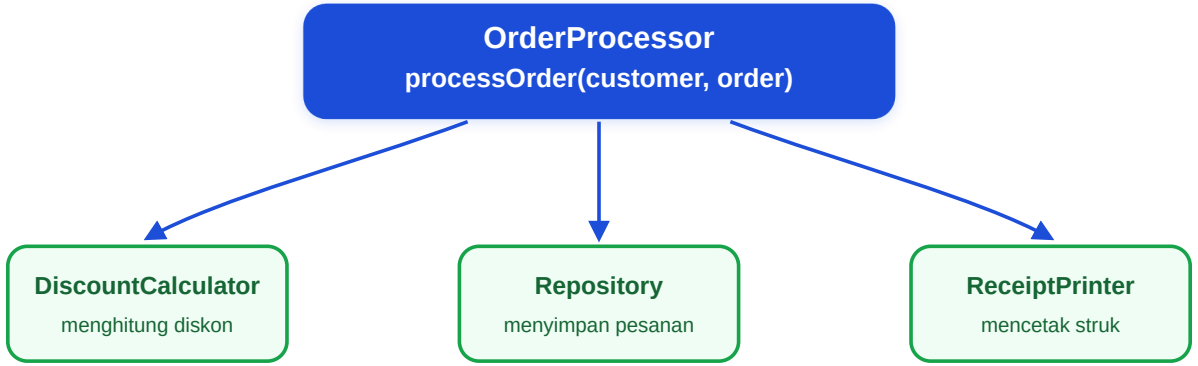
`Shippable`: LSP dan ISP Sekaligus



`Shippable` hanya punya `ship()` (Interface Segregation).

`PhysicalOrder` `implements Shippable`; `DigitalOrder` sengaja tidak, sebab pesanan digital tidak pernah dikirim. Tidak ada `ship()` palsu (Liskov Substitution).

`OrderProcessor`: Single Responsibility



OrderProcessor hanya mengoordinasikan,
tiap tanggung jawab didelegasikan ke kelasnya sendiri

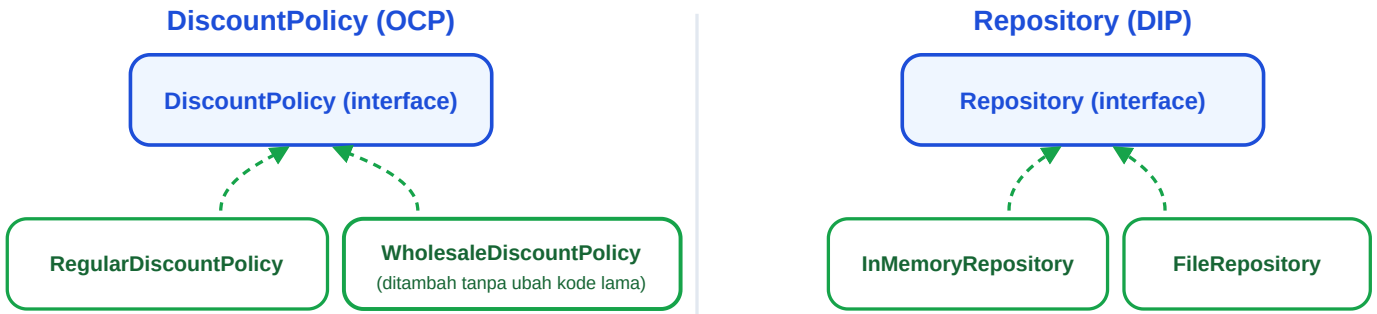
`OrderProcessor` tidak menghitung, tidak menyimpan, tidak mencetak. Tiga tugas itu diserahkan ke `DiscountCalculator`, `Repository`, dan `ReceiptPrinter`.

Contoh Kode: `OrderProcessor` Hanya Mengoordinasikan

```
public void processOrder(Customer customer, Order order) {
    int discount = discountCalculator.calculate(customer, order);
    int total = order.getAmount() - discount;
    repository.save(order, total);
    receiptPrinter.print(customer, order, discount, total);
}
```

Hitung diskon, simpan, cetak struk. Masing-masing oleh kelas lain.

`DiscountPolicy` dan `Repository`: OCP dan DIP



keduanya sama-sama pola "banyak implementasi kecil, satu interface": OCP untuk memilih perilaku, DIP untuk memilih penyimpanan

Pola yang sama: satu interface, banyak implementasi kecil. Jenis diskon baru cukup kelas baru (Open/Closed). `OrderProcessor` hanya mengenal `Repository`, jadi penyimpanan bebas ditukar (Dependency Inversion).

Contoh Kode: Menambah `DiscountPolicy` Baru

```
public class WholesaleDiscountPolicy implements DiscountPolicy {
    public int calculate(int amount) {
        return amount * 20 / 100;
    }
}
```

`DiscountCalculator` dan `OrderProcessor` tidak berubah. `DiscountCalculator` memegang `Map<String, DiscountPolicy>`, persis contoh di Bagian 3.

Kesalahan Umum: Memaksakan Kontrak yang Tidak Relevan

> **Salah:** `class DigitalOrder extends Order implements Shippable { public String ship() { return "N/A"; } }`, supaya "konsisten".

Benar: `class DigitalOrder extends Order { }` tanpa `Shippable`. `"N/A"` adalah janji palsu (Liskov), dan method yang tidak relevan dipaksakan (Interface Segregation).

Latihan

Toko ingin menambah `StudentDiscountPolicy` (potongan 10% untuk mahasiswa).

1. Berapa kelas baru yang perlu ditambah?
2. Perlukah `DiscountCalculator` atau `OrderProcessor` diubah?
3. Prinsip SOLID mana yang membuat hal itu mungkin?

Jawaban Latihan

1. **Satu kelas baru:** `StudentDiscountPolicy implements DiscountPolicy`.
2. **Tidak.** Kelas baru didaftarkan di `main`, bukan di dalam keduanya.
3. **Open/Closed.** Diperluas lewat kelas baru, kode lama tidak diubah.

Rangkuman Bagian 4

- `Shippable` kecil: Interface Segregation, dan menjaga Liskov.
- `OrderProcessor` hanya mengoordinasikan: Single Responsibility.
- `DiscountPolicy` : Open/Closed. `Repository` : Dependency Inversion.

Rangkuman Pertemuan 11

- SOLID: satu kelas satu tugas; kelas baru bukan ubah kode lama; pengganti harus tetap bisa; interface kecil; bergantung pada interface.
- `ArrayList` untuk data yang bertambah, `Map` untuk cari lewat kunci.
- Sistem pemrosesan pesanan menunjukkan kelima prinsip bekerja bersama.

Referensi

Deitel, *Java How to Program*, bab Collections; Martin, *Agile Software Development* (SOLID Principles)

Oracle Java Tutorials: "The Collections Framework"

Latihan pemrograman untuk materi ini tersedia di jobsheet Praktikum Pemrograman Berbasis Objek (RTI253008), Pertemuan 11

Tugas 1: Single Responsibility

Kode saat ini: satu kelas mengerjakan empat tugas.

```
class OrderService {
    boolean validate(Order order) { /* ... */ return true; }
    int shippingCost(Order order) { /* ... */ return 0; }
    void saveToFile(Order order) { /* ... */ }
    void sendConfirmationEmail(Order order) { /* ... */ }
}
```

1. Pecah menjadi empat kelas, satu tugas tiap kelas. Tuliskan nama kelas dan method utamanya.
2. Aturan ongkos kirim berubah. Kelas mana yang diubah sesudah dipisah?

Tugas 2: Open/Closed

Kode saat ini: tiap kurir baru menambah cabang `if`. Toko akan menambah `"SICEPAT"`.

```
class ShippingCalculator {
    int cost(Order order, String courier) {
        if (courier.equals("JNE")) { return 20000; }
        else if (courier.equals("POS")) { return 15000; }
        else if (courier.equals("GOJEK")) { return 25000; }   // cabang baru lagi
        return 0;
    }
}
```

1. Tuliskan `interface ShippingMethod { int cost(Order order); }` dan kelas `SicepatShipping`.
2. Tuliskan ulang `ShippingCalculator` agar menerima `ShippingMethod`, plus baris di `main` yang memilih `SicepatShipping`.

Tugas 3: Liskov Substitution

Kode saat ini: `Rectangle` punya `setWidth`, `setHeight`, dan `area()` = lebar x tinggi.

```
class Square extends Rectangle {
    void setWidth(int w)  { width = w; height = w; }    // ikut mengubah tinggi
    void setHeight(int h) { width = h; height = h; }
}

Rectangle r = new Square();
r.setWidth(4); r.setHeight(5);
System.out.println(r.area());
```

1. Berapa yang diharapkan pemanggil, berapa yang tercetak?
2. Ubah: `Square` tidak lagi `extends Rectangle`. Keduanya `implements Shape { int area(); }`. Tuliskan kedua kelas, gambar diagramnya di kertas.

Tugas 4: Interface Segregation

Kode saat ini: `RobotWorker` dipaksa mengisi method yang tidak masuk akal baginya.

```
interface Worker { void work(); void eat(); void sleep(); }

class RobotWorker implements Worker {
    public void work()  { /* ... */ }
    // robot tidak makan dan tidak tidur:
    public void eat()   { throw new UnsupportedOperationException(); }
    public void sleep() { throw new UnsupportedOperationException(); }
}
```

1. Pecah `Worker` menjadi interface kecil. Tuliskan deklarasinya.
2. Tuliskan ulang baris `implements` untuk `RobotWorker` dan `HumanWorker`, lalu hapus method yang tidak perlu.

Tugas 5: Dependency Inversion

Kode saat ini: terikat ke email. Toko ingin mengirim lewat WhatsApp juga.

```
class NotificationService {
    private EmailSender sender = new EmailSender();   // kelas sungguhan dibuat sendiri

    void notifyCustomer(String to, String text) {
        sender.sendEmail(to, text);
    }
}
```

1. Tuliskan `interface MessageSender { void send(String to, String text); }`, lalu `EmailSender` dan `WhatsAppSender` yang mengimplementasikannya.
2. Tuliskan ulang `NotificationService` agar menerima `MessageSender` lewat constructor, plus baris di `main` yang memilih WhatsApp.

Tugas 6: Collections

Kode saat ini: riwayat pesan disimpan di array berukuran 10.

```
class NotificationService {
    private String[] sent = new String[10];
    private int count = 0;

    void log(String to, String text) {
        sent[count++] = to + ": " + text;   // pesan ke-11 gagal
    }
}
```

1. Ganti array dengan koleksi yang bertambah otomatis. Tuliskan deklarasi field dan `log(...)` yang baru.
2. Riwayat harus bisa dicari lewat nomor penerima. Pilih `ArrayList` atau `Map`, tuliskan deklarasi field dan satu baris pencariannya.